

Assessment for AI Engineer

Thank you for your interest in joining our team. We appreciate the time and effort you've taken to apply. To proceed with the evaluation, please choose **one** of the tasks listed and complete it according to the instructions provided.

Our average review time is **7 days**, and we kindly ask that you **carefully follow all task guidelines** to ensure a smooth assessment process.

Test 1: Fine-Tuning TTS Model on Bangla Dataset

1. Project Overview

- Fine-tune a **Text-to-Speech (TTS) model** on the Bangla language dataset: [Comprehensive Bangla TTS](#)
- Candidate must choose the **best TTS model architecture** (e.g., Tacotron2, HiFiGan+FastSpeech2, VITS)
- Evaluate model performance using objective and subjective metrics

2. Requirements

2.1 Functional Requirements

- Preprocess Bangla text and normalize it for TTS
- Fine-tune TTS model using Kaggle free GPUs or local GPU
- Generate audio samples from test text
- Evaluate using:
 - **Objective metrics:** MOSNet, PESQ, MCD (Mel Cepstral Distortion)
 - **Subjective metrics:** human evaluation on clarity, naturalness, accent fidelity

Tables/Logs:

- **TTSExperiments:** `id`, `model_name`, `hyperparameters`, `train_loss`, `val_loss`, `objective_metrics`, `timestamp`

- `GeneratedSamples`: `id`, `experiment_id`, `text_input`, `audio_url`, `timestamp`

2.2 Core Design & Algorithm Requirements

- **OOP**: `TTSManager`, `Preprocessor`, `Evaluator`
- **Algorithm**:
 - Audio preprocessing, feature extraction (mel-spectrogram)
 - Model fine-tuning loop with logging
 - Performance evaluation and visualization
- **Design Pattern**: Strategy pattern for switching between TTS models

3. Non-Functional Requirements

- Use HuggingFace TTS pipelines or other open-source frameworks
- Modular and configurable code for hyperparameters
- Logging, checkpointing, and reproducible experiments
- Clean Jupyter/Notebook demos for generated audio

4. Deliverables

- Notebook or Python scripts for preprocessing, fine-tuning, and inference
- Sample generated audios
- Table of evaluation metrics per experiment
- Documentation: model choice, hyperparameters, and performance analysis
- Video explanation of your submitted task uploaded in youtube or shareable public google drive. Share the link

Test 2: Fine-Tuning LLaMA

3.1-8B-Instruct on Bengali Empathetic Conversations

1. Project Overview

- Fine-tune **LLaMA 3.1-8B-Instruct** on [Bengali Empathetic Conversations](#)

- Candidate must use **LoRA or Unsloth** for efficient fine-tuning on free GPU (Kaggle)
- Sequence length **must not be reduced**

2. Requirements

2.1 Functional Requirements

- Preprocess dataset for LLM fine-tuning
- Fine-tune LLaMA 3.1-8B-Instruct using **parameter-efficient fine-tuning (LoRA/Unsloth)**
- Evaluate using:
 - **Perplexity**, BLEU, ROUGE
 - Human evaluation on **empathetic response quality**
- Store logs:
 - `LLAMAEperiments: id, model_name, lora_config, train_loss, val_loss, metrics, timestamp`
 - `GeneratedResponses: id, experiment_id, input_text, response_text, timestamp`

2.2 Core Design & Algorithm Requirements

- **OOP:** `LLAMAFineTuner`, `DatasetProcessor`, `Evaluator`
- **Algorithm:**
 - LoRA adaptation for attention layers
 - Full-sequence tokenization (sequence length not reduced)
 - Evaluation pipeline for metrics + human testing
- **Design Pattern:** Strategy pattern for choosing LoRA or Unsloth
- **Resource optimization:** Split large model into gradient checkpointing, mixed precision training

3. Non-Functional Requirements

- Use Kaggle free GPU environment efficiently
- Logging and checkpointing for reproducibility
- Modular code for swapping datasets, LoRA configs, evaluation metrics

4. Deliverables

- Scripts or notebooks for preprocessing, LoRA/Unsloth fine-tuning, evaluation
- Sample model responses on test prompts
- Evaluation metrics table and analysis
- Documentation: choice of LoRA/Unsloth configuration, training strategy, challenges faced
- Video explanation of your submitted task uploaded in youtube or shareable public google drive. Share the link

Test 3: Fine-Tuning BLIP/CLIP for Visual Question Answering

1. Project Overview

- Fine-tune **BLIP** or **CLIP** models on [Visual Question Answering](#) dataset
- Candidate must implement **multimodal input processing** (image + text question → answer)

2. Requirements

2.1 Functional Requirements

- Preprocess image and text question pairs
- Fine-tune BLIP/CLIP for VQA task
- Evaluate using:
 - Accuracy, F1-score, BLEU
 - Human evaluation for answer correctness
- Store logs:
 - `VQAExperiments: id, model_name, hyperparameters, train_loss, val_loss, metrics, timestamp`
 - `GeneratedAnswers: id, experiment_id, image_url, question, answer, timestamp`

2.2 Core Design & Algorithm Requirements

- **OOP:** `VQAManager`, `DatasetProcessor`, `Evaluator`
- **Algorithm:**
 - Image encoding (BLIP/CLIP vision encoder)
 - Text encoding (question embedding)
 - Multimodal fusion for answer prediction
- **Design Pattern:** Strategy pattern to swap between BLIP and CLIP
- **Performance:** Batch processing, GPU optimization, caching embeddings

3. Non-Functional Requirements

- Modular code for switching datasets, models, and fusion techniques
- Logging, checkpointing, and reproducible experiments
- Efficient preprocessing of large image datasets

4. Deliverables

- Notebook or Python scripts for preprocessing, fine-tuning, and inference
- Sample VQA predictions on test set
- Evaluation metrics table and visualizations
- Documentation: model choice, hyperparameters, performance evaluation
- Video explanation of your submitted task uploaded in youtube or shareable public google drive. Share the link